

Stockage des données

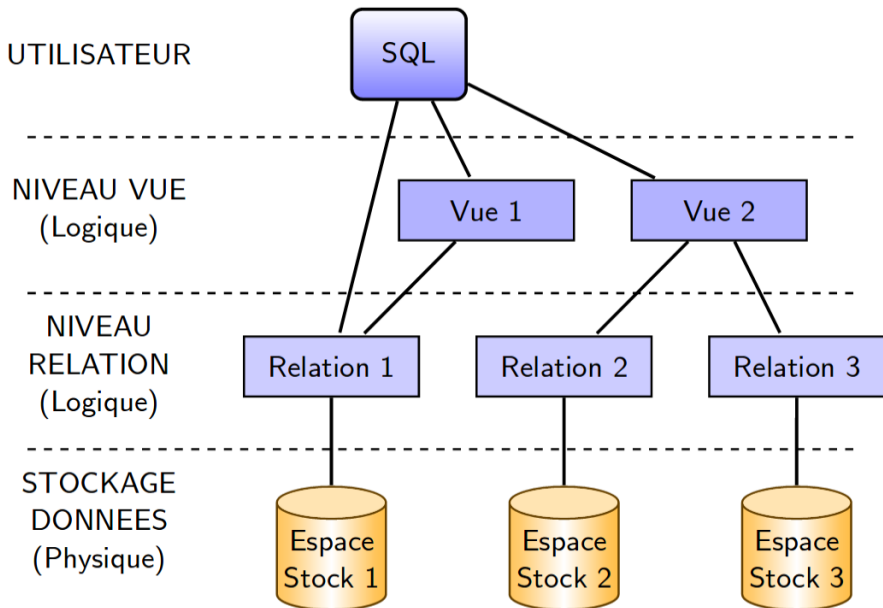
Géraldine Del Mondo

Département Informatique et Technologies de l'Information

Institut National des Sciences Appliquées - Rouen

geraldine.del_mondo@insa-rouen.fr

Différents niveaux d'une BD



Les objets à stocker

- **Les relations système**
 - ↳ Description de la base
 - ↳ BD système
- **Les relations utilisateur**
 - ↳ Données utilisateur
- **Les index**
 - ↳ Sur les relations utilisateur
(accélérer les accès aux données)

Avant d'aller plus loin...

Et pourquoi pas tout en
séquentiel ?

Organisation du stockage séquentiel... et plus.

Relation Personnes

Id	Nom	AnneeNaiss	Adresse
3	A	1950	...
2	R	1980	...
4	C	2000	...
1	B	1968	...

3	A	1950	...	2	R	1980	...	4	C	2000	...	1	B	...
---	---	------	-----	---	---	------	-----	---	---	------	-----	---	---	-----

1	B	1968	...	2	R	1980	...	3	A	1950	...	4	C	...
---	---	------	-----	---	---	------	-----	---	---	------	-----	---	---	-----

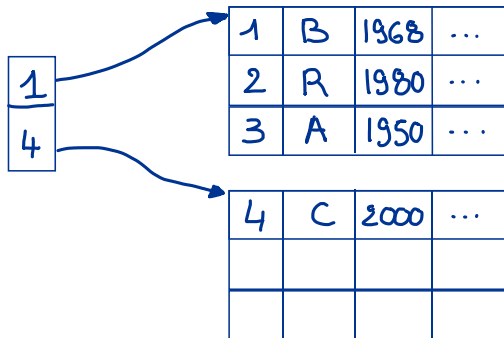
3	A	1950	...	1	B	1968	...	4	C	2000	...	2	R	...
---	---	------	-----	---	---	------	-----	---	---	------	-----	---	---	-----

Organisation du stockage indexation...

Relation Personnes

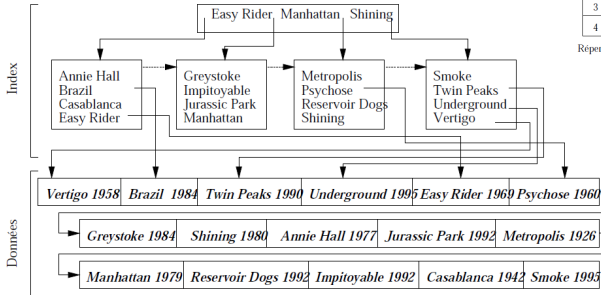
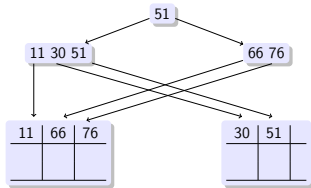
Id	Nom	AnneeNaiss	Adresse
1	B	1968	...
2	R	1980	...
3	A	1950	...
4	C	2000	...

(↑ Vue logique)



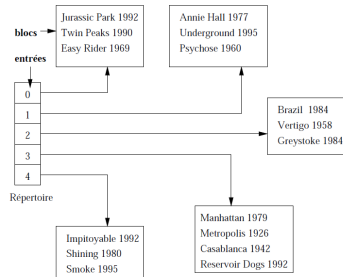
← index → données →

Exemples d'index



Taille d'un bloc = 4

- $h(\text{titre}) = \text{rang}(\text{titre}[0] \bmod 5)$
 Impitoyable $\rightarrow$ 'I' $\rightarrow 9, 9 \bmod 5 = 4$



BD : niveau Logique / Physique

BD

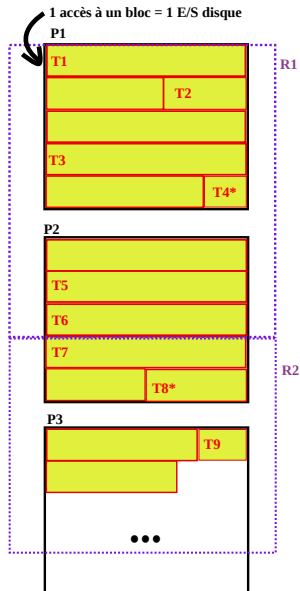
- Logique : ensemble de relations
- Physique : ensemble de pages (extensible)
- Notion de **page** : Un **bloc** de l'espace de stockage (unité d'entrée/sortie)

Relation

- Logique : ensemble de tuples
- Physique : ensemble de pages de la BD physique
- Un fichier physique

Tuple

- Logique : ensemble de valeurs d'attributs
- Physique : suite d'octets contenue dans une même page (hypothèse)
- Enregistrement fichier



$$|P1| = |P2| = |P3|$$

Relation = l'ensemble de bloc(s) = 1 fichier physique

1 tuple = 1 suite d'octets contigus dans un même bloc (hypothèse, * Ici T4 et T8 sont segmentés)
= 1 enregistrement

Questions

- Comment je **range** mes données ?
- Comment j'**accède** à mes données ?

Volume de données

Types de requêtes

Le besoin

- **Ranger** : méthodes d'organisation/de **placement**
- **Accéder** : méthodes de recherche/d'**accès**

Questions

- Comment je **range** mes données ?
- Comment j'**accède** à mes données ?

Volume de données

Types de requêtes

Le besoin

- **Ranger** : méthodes d'organisation/de **placement**
- **Accéder** : méthodes de recherche/d'**accès**

Objectifs des méthodes de placement et d'accès

- **Limitation des accès aux seules données pertinentes afin de minimiser le nombre d'E/S disque**
- **Adaptation à la croissance du volume des données de la base**

Concepteur BD

→ choix d'une méthode équilibrée entre le temps mis pour accéder aux données et l'espace mémoire utilisé. Il fera aussi attention également au temps pris par les m.a.j des structures induites par la méthode choisie.

Organisation/Placement

💡 Définition PLACEMENT

Méthode d'organisation (rangement) des tuples d'une relation sur disque en fonction des valeurs d'un attribut ou d'un groupe d'attributs.

Méthode de placement possibles

Séquentiel non trié

Séquentiel trié

Indexé

Par hachage

Caractéristiques pouvant être appliquées

Mono-attribut / Multi-attributs

Clé primaire ou non

Comparaison de clés

Calcul de clé

Statique ou dynamique

Recherche/Accès



Définition ACCÈS

Méthode de recherche d'un tuple ou d'un groupe de tuples d'une relation en fonction de la valeur d'un attribut ou d'un groupe d'attributs.

Méthode d'accès possibles

Séquentiel

Sélectif (direct)

Caractéristiques pouvant être appliquées

Tous les tuples
Certains tuples

Mono-attribut / Multi-attributs
Clé primaire ou non
Valeur exacte ou intervalle

Ordre quelconque
Critère de tri

Notez bien

Il ne faut pas confondre l'**organisation** des données qui répond à des méthodes basées sur des caractéristiques de **placement**, et l'**accès** aux données qui fait référence à des méthodes basées sur des caractéristiques de **sélection**.

Pour une donnée :

- Il n'y a qu'une seule méthode de placement
- Il y a potentiellement plusieurs méthodes d'accès

CONCLUSION INTERMÉDIAIRE

Conclusion

Nous avons vu...

- la différence entre le placement et l'accès aux données
- les objectifs des méthodes de placement et d'accès aux données

Nous pouvons donc aborder...

- en détail les méthodes de placement et d'accès...
...commençons par regarder sur deux exemples ce qu'on entend par indexation

ARBRE B & CIE

- ➔ **Objectif** : utiliser des structures de données spécifiques pour accélérer l'accès aux données
- ➔ Ces structures peuvent aussi bien permettre le placement ou l'accès aux données

Exemple introductif [1]

Exemple support[1] Données

titre	année	...
Vertigo	1958	...
Brazil	1984	...
Twin Peaks	1990	...
Underground	1995	...
Easy Rider	1969	...
Psychose	1960	...
Greystoke	1984	...
Shining	1980	...
Annie Hall	1977	...
Jurassic Park	1992	...
Metropolis	1926	...
Manhattan	1979	...
Reservoir Dogs	1992	...
Impitoyable	1992	...
Casablanca	1942	...
Smoke	1995	...

Exemple support[1] requêtes potentielles

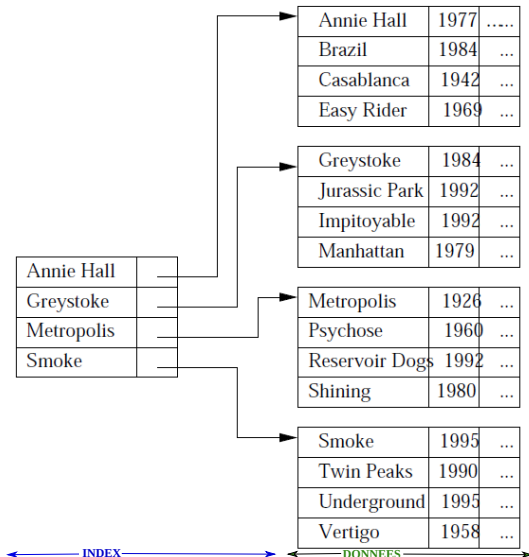
```
-- sur un attribut
SELECT *
FROM Film
WHERE titre = 'Vertigo';
```

```
-- sur plusieurs attributs
SELECT *
FROM Film
WHERE nom = 'Hitchcock'
      AND prenom = 'Alfred';
```

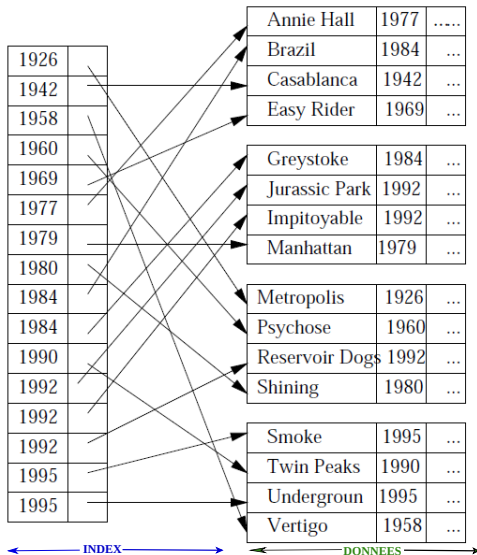
```
-- intervalle
SELECT *
FROM Film
WHERE annee BETWEEN 1960
                  AND 1980;
```

```
-- préfixe
SELECT *
FROM Film
WHERE titre LIKE 'M%';
```

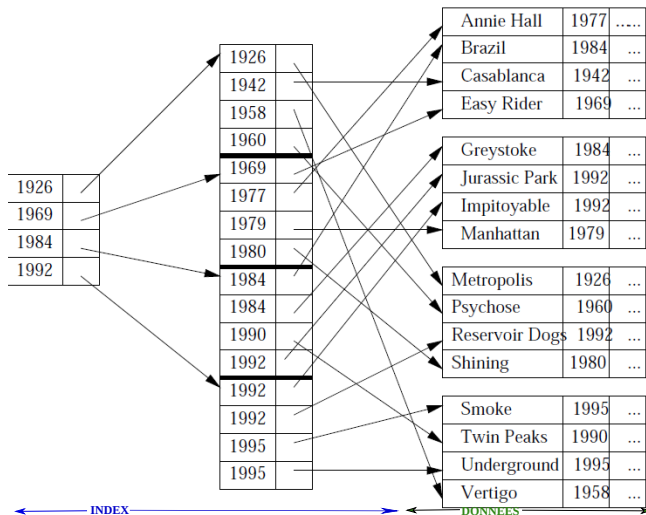
Exemple support[1] Index non dense



Exemple support[1] Index dense



Exemple support[1] Index multi-niveaux non plaçant



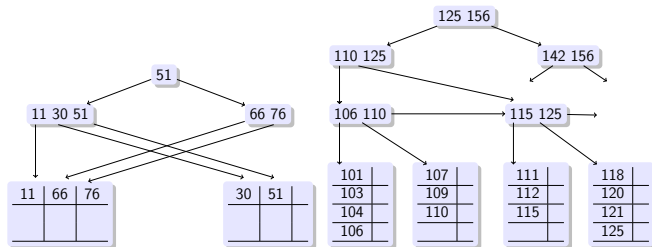
💡 Définition INDEX DENSE

Si toutes les valeurs de clé sont représentées dans l'index alors il est dense.

💡 Définition INDEX PLAÇANT

Si les enregistrements associés aux clés sont “classés/rangés” dans l'ordre des clés (adressage dans un même bloc ou bloc contigus de données) alors l'index est plaçant .

Un index non plaçant est forcément dense.



► et donc un index c'est quoi ?

Arbre B+

Quelle différence entre un arbre B et B+ ?

► au fait c'est quoi déjà un arbre B ? (+cf. cours Algo ASI3)

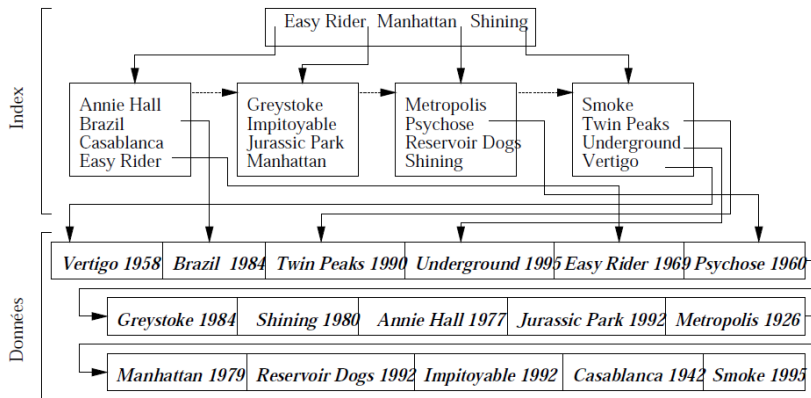
Dans un arbre B+ :

- les données se situent dans les feuilles (ou pointeurs sur ces données)
- les feuilles sont chaînées les unes aux autres
- l'opération d'insertion est similaire à celle dans un arbre B
- la recherche se termine au niveau des feuilles
- la suppression n'aura lieu que dans les feuilles
- meilleure efficacité dans la recherche du suivant (chaînage)

L'arbre B/B+ est un bon compromis, il est utilisé par défaut lors de l'indexation de la clé primaire des tables relationnelles.

► Quel sens pour une clé dans le contexte du stockage de données ?

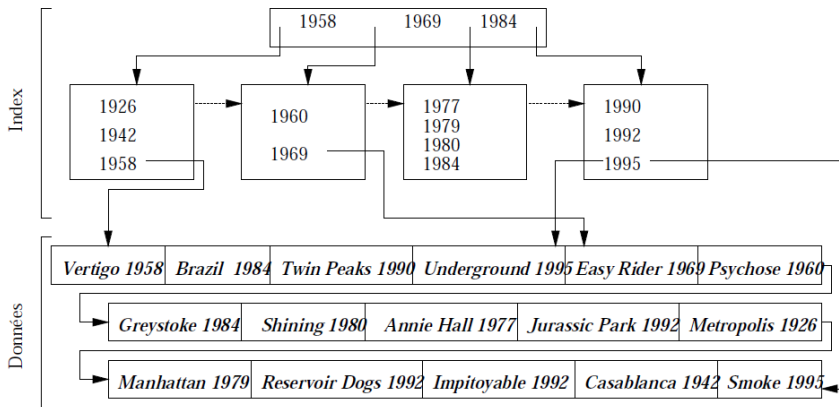
Exemple [1] en arbre B+ : index sur le titre



```
SELECT * FROM Film WHERE titre = 'Vertigo'; (2 + 1 E/S)
```

► Comment se mesure l'efficacité d'un index ?

Exemple [1] en arbre B+ : index sur l'année



```
SELECT * FROM Film WHERE annee BETWEEN 1960 and 1980 ;
(3 + 5 E/S)
```

► Index sur le titre, index sur l'année mais encore ?

Exemple [1] Requêtes 'préfixe/suffixe'

```
SELECT *  
FROM Film  
WHERE titre like 'M%';
```

```
SELECT *  
FROM Film  
WHERE titre like '%M';
```

► Comment se rendre compte concrètement de l'intérêt des index ?

Organisations spécifiques : Vocabulaire (pour info)

- ISAM : index trié, plaçant, non dense, (statique → zone de débordement), index en arbre, données triées
- VSAM : index trié, plaçant, non dense, index et données triées dans un unique arbre B+
- IS3 : index trié, dense, index en arbre B+, données non triées

CONCLUSION INTERMÉDIAIRE : ARBRE B+

Conclusion sur l'utilisation des arbres B+

Avantages

- Supporte les recherches par clé, préfixe et intervalle
- Complexité recherche

Inconvénients

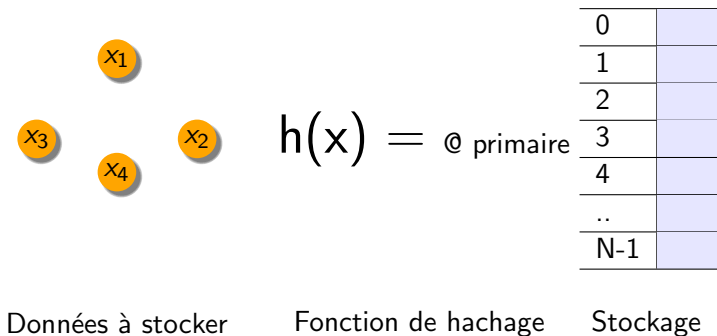
- Complexité insertion/suppression
- Taille de l'index

► [Lien vers la conclusion optimisation des requêtes](#)

Hachage

Le Hachage : principe (1)

→ Permettre l'amélioration des performances d'accès aux données



Le Hachage : principe (2)

$$h(x_4) = h(x_2)$$

Collision

0	x_1
1	
2	
3	x_2
4	x_3
..	
N-1	

x_4 va être placé à un autre emplacement : en **débordement** $\Rightarrow$
 Algorithme de résolution de collision

Le Hachage

Définition HACHAGE (BD)

- Division d'un fichier en paquet de taille fixe
- A l'intérieur d'un paquet les tuples ne sont pas triés
- La clé est convertie en un numéro de paquet à l'aide d'une fonction de hachage

Méthode

1. Considérer la valeur de la clé comme une suite de bits
2. Appliquer une fonction à valeur dans $[0, N-1]$
(N = nombre de paquets primaires)

Exemples de fonctions de hachage : Extraction, pliage, division etc.

Caractéristique d'une bonne fonction de hachage

Elle doit :

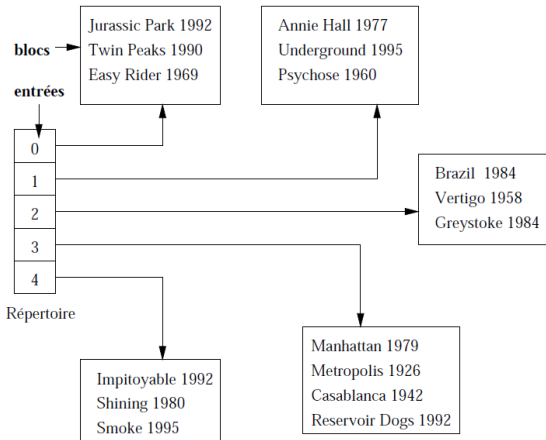
- Etre calculable rapidement
- Assurer une répartition uniforme

Exemple hachage [1]

Taille d'un bloc = 4

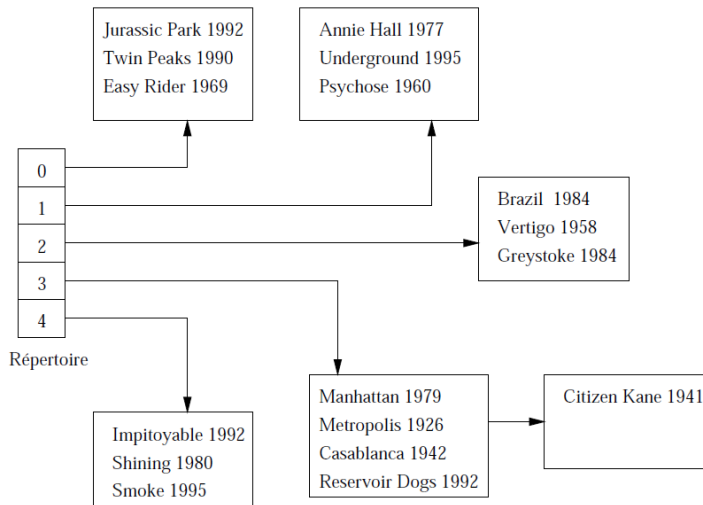
- $h(\text{titre}) = \text{rang}(\text{titre}[0] \bmod 5)$

Impitoyable $\rightarrow$ 'I' $\rightarrow 9, 9 \bmod 5 = 4$



$\rightarrow$ Ajout de 'Citizen Kane 1941' ?

Hachage statique



➔ **Débordement !**

► Qu'est-ce qu'un index statique ?

Bilan hachage statique

Avantages

- En absence de débordement, accès sur clé de placement très performant
- Bien adapté pour des relations de taille moyenne à faible croissance

Inconvénients

- Pas d'accès séquentiel trié
- Pas de question d'intervalle
- Paquets primaires pouvant être vides
- Un paquet peut devenir plein
- Résolutions des collisions par gestion de paquets de débordement (e.g. Chaînage séparé)
- Dégradation des performances de l'accès direct quand débordement
- Réorganisation périodique
- Mal adapté aux relations à forte croissance

⇒ Organisation par **hachage dynamique**

Hachage dynamique

Définition HACHAGE DYNAMIQUE

- Objectif : adaptation du hachage à la dynamique des fichiers
- Principe : changer de fonction de hachage de manière dynamique lors de la croissance du fichier, en utilisant progressivement la chaîne de bits résultant de l'application de la fonction de hachage.

- Hachage dynamique
- Hachage virtuel
- **Hachage extensible**
- Hachage linéaire

Hachage extensible

Définition HACHAGE EXTENSIBLE

Structure :

- Un catalogue dynamique contenant les adresses des paquets
- Des paquets de hachage contenant les données et croissant dynamiquement

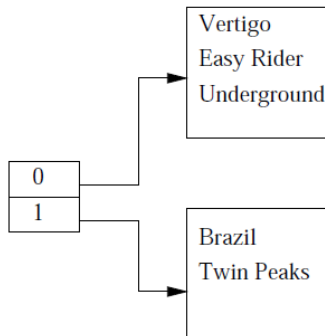
Principes :

- Utilisation progressive du hash-code de N bits
- La fonction de hachage adresse un catalogue contenant les adresses des paquets
- Taille du catalogue $= 2^M$
- Extension du fichier quand un paquet devient plein par éclatement de ce dernier

Exemple Hachage [1] Fonction de hachage

titre	$h(\text{titre})$
Vertigo	01110010
Brazil	10100101
Twin Peaks	11001011
Underground	01001001
Easy Rider	00100110
Psychose	01110011
Greystoke	10111001
Shining	11010011

Exemple Hachage [1] (taille bloc = 3)

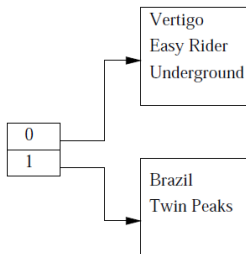


Ajout de 'Psychose'

01110011

$h'(1) = 0$

Exemple Hachage [1] (taille bloc = 3)



Ajout de 'Psychose'

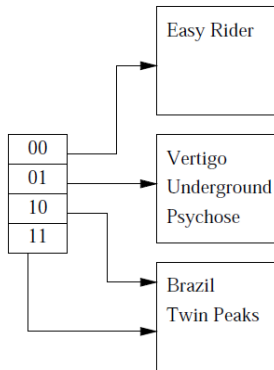
01110011 $h(2) = 01$

'Easy Rider' $h(2) = 00$

'Brazil' $h'(2) = 10$

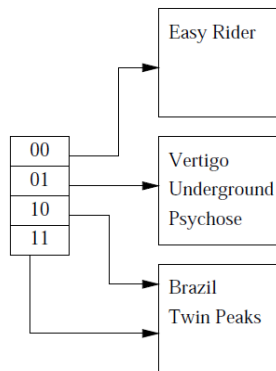
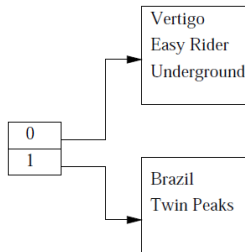
'Twin Peaks' $h'(2) = 11$

'Vertigo' $h'(2) = 01$



Doublment du répertoire !

Exemple Hachage [1] (taille bloc = 3)



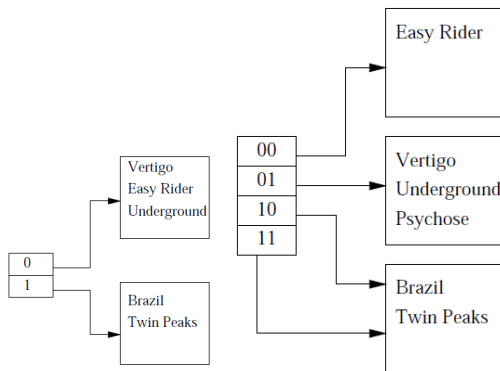
Ajout de ' Greystoke' :

10111001 $h'(2) = 10$

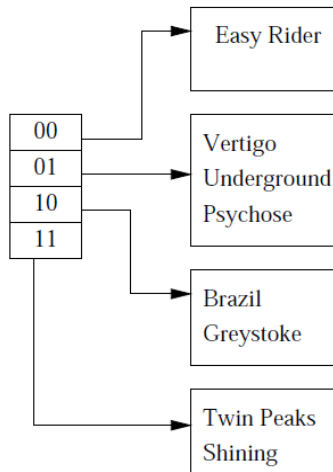
puis 'Shinning' : 11010011

$h'(2) = 11$

Exemple Hachage [1] (taille bloc = 3)



Ajout de 'Greystoke' :
 10111001 $h'(2) = 10$
 puis 'Shinning' :
 11010011 $h'(2) = 11$



Réorganisation !

Bilan hachage extensible

Avantages

- Accès sélectif sur clé de placement toujours performant
- Adapté à des relations à forte dynamicité

Inconvénients

- Croissance exponentielle de la taille du catalogue
- Pas d'accès séquentiel trié
- Pas de question d'intervalle

Conclusions

Conclusion sur l'utilisation du hachage dynamique par rapport aux arbre-B

+ rapide qu'un arbre-B ...

hachage : $O(1)$

B-Tree : $O(\log(n))$

... quand c'est possible de l'utiliser

- les données ne sont pas triées : pose entre autre problème pour répondre à des requêtes qui renvoient les données triées
- les opérateurs disponibles sont limités à '=' et '<>' (arbre-B : '=', '<>', '<=', '>=', between)

Remarque : Le Hachage est une structure plaçante, on ne peut définir qu'une seule table de hachage pour un ensemble de données (arbres B/B+ supplémentaires si nécessaire)

► [Lien vers la conclusion générale](#)

PLACEMENT & ACCÈS

La “jungle” des clés

Définition CLÉS

Clé : un attribut ou un groupe d'attributs

Clé primaire relationnelle

- Niveau Logique
- Identifie un tuple de manière unique dans une relation
- Une seule clé primaire, pour potentiellement de 0 à N clés candidates

Clé de placement

- Niveau physique
- Placement d'une relation
- Clé d'accès primaire
- Une seule clé de placement

Clé d'accès secondaire

- Niveau physique
- Définit un chemin d'accès secondaire
- 0 à N clés d'accès secondaires

Les index

Objectif

Limiter l'accès aux seules blocs/pages utiles d'une relation

- **Implantation** : Relation de schéma (attribut_index, TID)
- **Gestion** : Assurée par le SGBD
- **Caractéristiques** : Trié, Plaçant/Non plaçant, Dense/Non dense

Avantage

- Accélération de certaines requêtes

Inconvénients

- Stockage des index
- Mise à jour des index lors de la mise à jour des tables concernées

EXEMPLE index : Arbre-B

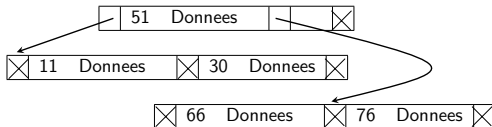
Définition ARBRE-B

Un arbre-B d'ordre m est un arbre tel que :

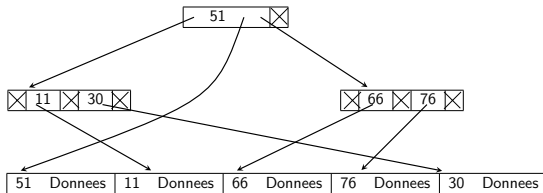
- ① Chaque nœud contient k clés triées avec : $m \leq k \leq 2m$ (nœud non racine) et $1 \leq k \leq 2m$ (nœud racine).
- ② Chaque chemin de la racine à une feuille est de même longueur à 1 près
- ③ Un nœud est :
 - Soit terminal (feuille)
 - Soit possède $(k + 1)$ fils tels que les clés du i ème fils ont des valeurs comprises entre les valeurs du $(i - 1)$ ème et i ème clés du père

EXEMPLE Index arbre-B

Organisation en arbre B (placement)



Indexation en arbre B (accès)

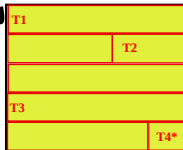


[◀ retour à l'exemple](#)

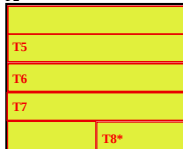
Efficacité d'un index

→ Faut-il toujours mieux utiliser un index ?

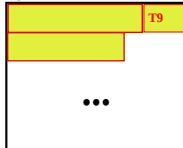
1 accès à un bloc = 1 E/S disque
P1



P2



P3



$|P1| = |P2| = |P3|$

Balayage : Le temps de balayage est proportionnel à la taille du fichier

Tps moyen d'accès à un enregistrement =

$$\frac{\text{Nb de blocs} \times \text{Tps moyen d'accès à un bloc}}{\text{Nb d'enregistrements recherchés}}$$

Index : Le temps d'accès est proportionnel au nombre d'enregistrements

Tps moyen d'accès à un enregistrement
= Nombre de niveaux de l'index $\times$ Tps
moyen d'accès à un bloc

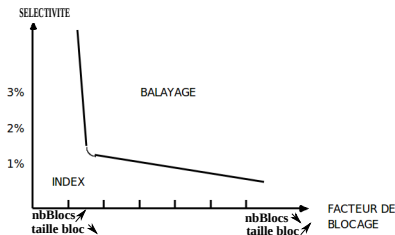
→ Si le nombre de niveaux dans l'index est trop grand, l'index perd en efficacité

Efficacité d'un index (2)

💡 Définitions

facteur de blocage = $\frac{\text{nb total d'enreg.}}{\text{Nb de blocs}}$: nombre d'enreg./bloc

sélectivité = $\frac{\text{Nb d'enreg. recherchés}}{\text{nb total d'enreg.}}$: proportion des enregistrements recherchés au regard de la totalité des articles



◀ retour à l'exemple

💡 Autrement dit :

→ si on a pas un taux de sélectivité suffisant, l'utilisation d'index est inutile (et potentiellement pénalisante)

Et donc...

Comment se rendre compte
concrètement de l'intérêt des
index ?

Plan d'exécution [2]

Définition PLAN D'EXÉCUTION

Le plan d'exécution d'une requête est ce qui est effectivement réalisé par le SGBD pour y répondre. Il est défini par l'optimiseur de requête qui détermine les étapes permettant d'exécuter la requête le plus efficacement possible.

EXEMPLE

EXPLAIN SELECT * FROM ANIMAUX ;

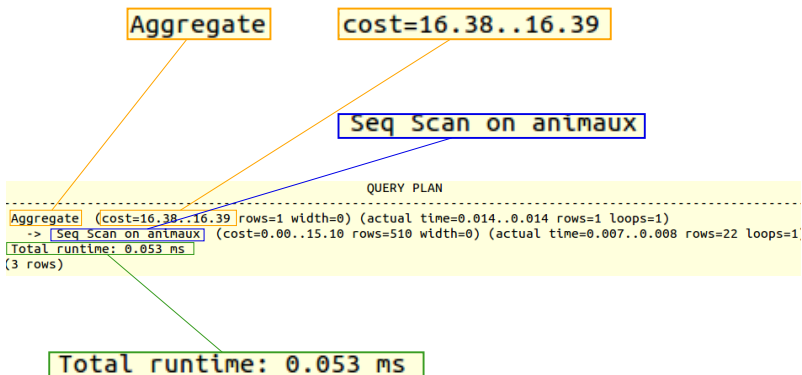
QUERY PLAN

Seq Scan on animaux (cost=0.00..15.10 rows=510 width=128)
(1 row)

➔ Attention, toutes les valeurs fournies sont des **estimations** !

Plan d'exécution (EXEMPLE 1)

EXPLAIN ANALYZE SELECT COUNT(*) FROM ANIMAUX ;



Plan d'exécution (EXEMPLE 2)

EXPLAIN ANALYZE SELECT noveto,
espece, COUNT(noeanim) AS nb
FROM animaux, veto
GROUP BY espece, noveto
HAVING compespece=espece
ORDER BY noveto;

GroupAggregate

Sort

Sort Key: veto.noveto, animaux.espece

QUERY PLAN

```
GroupAggregate (cost=94.05..108.85 rows=740 width=124) (actual time=0.111..0.121 rows=7 loops=1)
-> Sort (cost=94.05..95.90 rows=740 width=124) (actual time=0.104..0.104 rows=20 loops=1)
    Sort Key: veto.noveto, animaux.espece
    Sort Method: quicksort Memory: 26kB
    -> Hash Join (cost=16.52..58.79 rows=740 width=124) (actual time=0.030..0.038 rows=20 loops=1)
        Hash Cond: ((animaux.espece)::text = (veto.compespece)::text)
        -> Seq Scan on animaux (cost=0.00..15.10 rows=510 width=86) (actual time=0.006..0.009 rows=22 loops=1)
        -> Hash (cost=12.90..12.90 rows=290 width=86) (actual time=0.010..0.010 rows=7 loops=1)
            Buckets: 1024 Batches: 1 Memory Usage: 1kB
            -> Seq Scan on veto (cost=0.00..12.90 rows=290 width=86) (actual time=0.003..0.003 rows=7 loops=1)
    Total runtime: 0.198 ms
    (11 rows)
```

Join

Hash Cond: ((animaux.espece)::text = (veto.compespece)::text)

Remarque TP

QUERY PLAN

```
Seq Scan on film (cost=0.00..33.44 rows=1844 width=30) (actual time=0.007..0.174 rows=1844 loops=1)
Planning time: 0.266 ms
Execution time: 0.270 ms
(3 rows)
```

Planning time = Trouve la meilleure requête

Execution time = exécute le plan + retourne le résultat

[◀ retour à l'exemple](#)

POINT SUR LE PLACEMENT (ORGANISATION)

Placement d'une relation

Placement d'une relation sur disque en fonction des valeurs d'un attribut ou d'un groupe d'attributs

Relation placée :

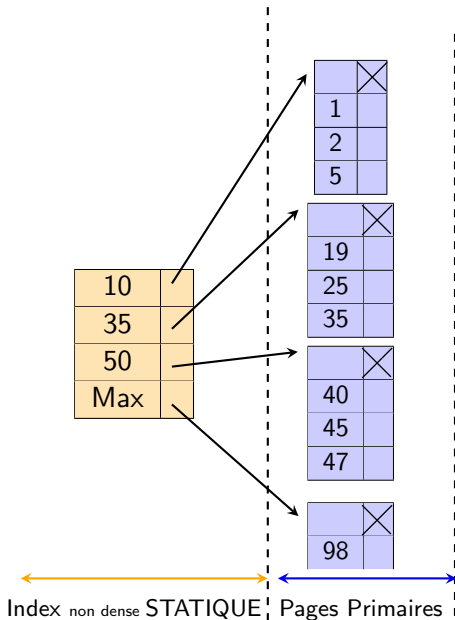
- Séquentiel trié
- Indexé
 - Statique (ISAM)
 - Dynamique : B-Tree (VSAM)
- Haché
 - Statique
 - Dynamique

Relation non placée

- Les tuples sont rangés dans l'ordre d'insertion
- Séquentiel non trié

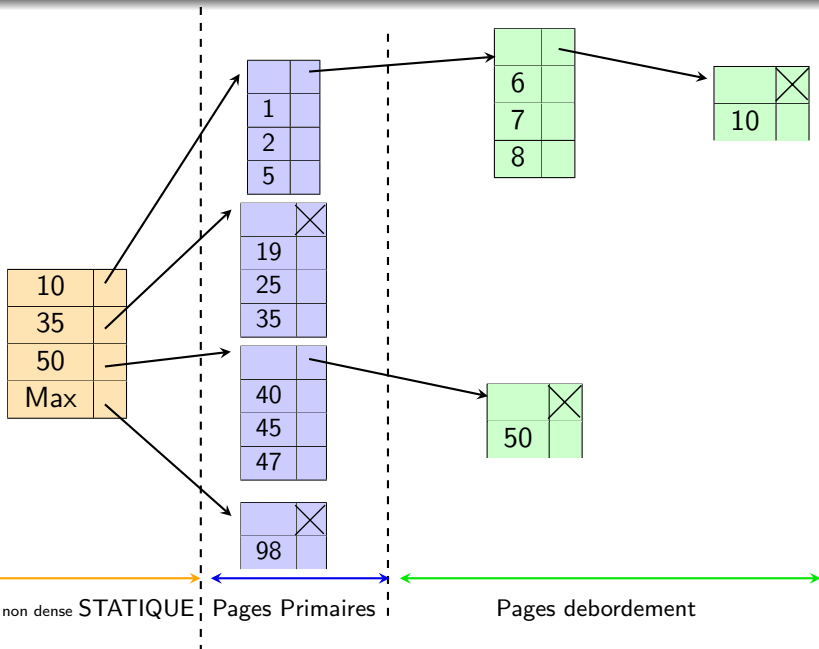
Exemple : Placement indexé statique

(état initial)



taille bloc = 4

Exemple : Placement indexé statique



Avantages

- Accès séquentiel trié et sélectif sur la clé de placement
- Accès sélectif performant si non débordement

Inconvénients

- Recherche non performante en zone de débordement
- Mise à jour couteuse en zone de débordement
- Nécessité de réorganisations périodiques
- Organisation mal adaptée pour les fichiers à forte croissance

⇒ **Organisation avec un index dynamique (e.g. Arbre-B)**

Extension du placement classique : multi-attributs

- Clé de placement = concaténation des attributs de recherche
- Définition relation d'ordre

EXEMPLE :

- Clé de placement : (CRU, MIL)
- Relation d'ordre : $(CRU_1, MIL_1) < (CRU_2, MIL_2)$
 $\iff CRU_1 < CRU_2 \text{ et } MIL_1 < MIL_2$

Critique

- Accélère les requêtes portant sur CRU et MIL
- Pas de restriction du champ de recherche pour les requêtes portant uniquement sur MIL

POINT SUR L'ACCÈS (RECHERCHE)

Les chemins d'accès primaires

Définition ACCÈS PRIMAIRE

Accès associé au placement d'une relation

EXEMPLE : Relation R placée sur un attribut A

- Placement séquentiel trié
 - ↪ Séquentiel trié sur A
- Placement indexé
 - ↪ Séquentiel trié sur A
 - ↪ Direct sur A via l'index
- Placement par hachage
 - ↪ Direct sur A via la fonction de hachage

Les accès secondaires

Hypothèse : Relation R placée sur un attribut A

Problème

- Favorise les accès sur A
- ↪ Défavorise les accès $\neq A$

Solution

- Création d'un chemin d'accès alternatif pour accélérer les accès sur un autre attribut B : un **index secondaire** sur B

Index secondaire

Définition INDEX SECONDAIRE

- Associe à chaque valeur V de l'attribut indexé les tuples de R ayant V comme valeur de l'attribut B
- Relation de schéma (Attribut_indexé, Identifiant_tuple)
En général implantée comme un arbre- B
- Index qui peut-être non discriminant

Relation R

Etu	nomEtu
111	Zaber
222	Habiche
333	Madelin
444	Briochin

Index Primaire

Etu	@Tuple
111	@Tuple1
222	@Tuple2
333	@Tuple3
444	@Tuple4

Index Secondaire

nomEtu	@Tuple
Briochin	@Tuple4
Habiche	@Tuple2
Madelin	@Tuple3
Zaber	@Tuple1

[◀ retour à l'exemple](#)

Index secondaire

Définition INDEX SECONDAIRE

- Associe à chaque valeur V de l'attribut indexé les tuples de R ayant V comme valeur de l'attribut B
- Relation de schéma (Attribut_indexé, Identifiant_tuple)
En général implantée comme un arbre- B
- Index qui peut-être non discriminant

Relation R

Etu	nomEtu
111	Zaber
222	Habiche
333	Madelin
444	Briochin

Index Primaire

Etu	@Tuple
111	@Tuple1
222	@Tuple2
333	@Tuple3
444	@Tuple4

Index Secondaire

nomEtu	@Tuple
Briochin	@Tuple4
Habiche	@Tuple2
Madelin	@Tuple3
Zaber	@Tuple1

[◀ retour à l'exemple](#)

CONCLUSION INTERMÉDIAIRE : OPTIMISATION

Optimisation (de requêtes)

Objectifs

- Minimiser le nombre d'E/S disque , minimiser le temps CPU

Les moyens du SGBD

- Restructuration arbre algébrique
- Implantation efficace des opérations algébriques
- Méthodes d'accès performantes, (l'optimiseur de requêtes choisit le meilleur chemin d'accès)

Le rôle de l'administrateur

- Spécifier le placement initial
- Définir des chemins d'accès judicieux

L'utilisateur

- Ne se préoccupe (théoriquement) pas des chemins d'accès (travaille uniquement au niveau logique (ou au niveau vue))

Nous avons vu...

- la définition et la nécessité (**ou pas**) d'un index (sélectivité...)
- le placement des données
- l'accès aux données (primaire/secondaire)
- la notion d'optimisation de requêtes

Conclusion

Nous avons vu...

- la différence entre un index dense et non dense, plaçant et non plaçant
- la représentation des données indexées par arbre B+, Hachage
- les avantages et inconvénients de ces structures

et il existe aussi...

- plein d'autres types d'index (bitmap, R-tree...)
- deux cours disponibles sur moodle pour parfaire ses connaissances sur les arbres B et le hachage (statique/dynamique)
- dans la suite des slides, plus de détail sur le placement et l'accès.

Bibliographie

[1] Cours de bases de données Philippe Rigaux 2001 :

<http://dept25.cnam.fr/BDA/DOC/cbd.pdf>

[2] Plans d'exécution : <https://use-the-index-luke.com/sql/explain-plan/postgresql/operations>

<https://robots.thoughtbot.com/reading-an-explain-analyze-query-plan>